

Basic (your first FastAPI route) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper FastAPI: a typed reply, a quick in-process check, and routes grouped with `APIRouter`). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

A FastAPI route is a normal Python function with a decorator on top that tells FastAPI which URL and HTTP method should run it. `GET /health` means: when a client visits `/health` with a GET request, run this function and send back whatever it returns as the reply.

Things to use:

- `from fastapi import FastAPI`
- `app = FastAPI()` — one app object for your whole service.
- `@app.get("/health")` — the decorator, placed directly above your function.
- `return {"status": "ok"}` — a plain dict; FastAPI turns it into JSON for you.
- Run it with `uvicorn health_route_practice:app --reload` (from inside `practice/`), then hit it with `curl localhost:8000/health`.

Intermediate Version

`uvicorn health_route_practice:app` is worth reading literally: `health_route_practice` is the filename (no `.py`), `app` is the variable name you assigned `FastAPI()` to. Uvicorn imports that module and looks for that variable. `--reload` restarts the server automatically every time you save the file — useful while developing, never used in production.

The `-> dict` return type hint doesn't check anything at runtime. A Pydantic model used as `response_model` does: FastAPI checks your *return value* against it, and shows its shape in the automatic docs at `http://localhost:8000/docs`.

The exact pieces:

- `from pydantic import BaseModel` then `class HealthResponse(BaseModel): status: str`.
- `@app.get("/health", response_model=HealthResponse)` — the reply shape is now checked and documented.
- `from fastapi.testclient import TestClient` — calls your routes directly in the same Python process, no server running: `client = TestClient(app)`, then `response = client.get("/health")`, then `print(response.status_code, response.json())`. It needs the `httpx` package installed.

- `if __name__ == "__main__":` around the `TestClient` lines — so they run with `python health_route_practice.py`, but not when uvicorn imports the file.
- `from fastapi import APIRouter` — once you have more than one or two routes, put them on a `router = APIRouter()` in their own file, and add them to the app with `app.include_router(router)`. The Build Task's `routes.py` uses exactly this.

Difference between Basic and Intermediate: Basic names the plain idea and the tools for the smallest possible working route. Intermediate explains what those tools actually do (the uvicorn import string, the automatic `/docs`), upgrades the bare dict to a checked `HealthResponse`, adds a quick in-process check with `TestClient` instead of a manual `curl`, and groups routes with `APIRouter` — the same layout every later exercise and the Build Task use.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
create app = FastAPI()

route GET /health:
    return {"status": "ok"}

run: uvicorn health_route_practice:app --reload
```

Here's the whole thing — try running it and hitting it yourself:

```
# practice/health_route_practice.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

Expected output if you run `uvicorn health_route_practice:app --reload` then `curl localhost:8000/health`:

```
{"status": "ok"}
```

Intermediate Version

```
define HealthResponse(BaseModel): status: str

create app = FastAPI()

route GET /health, response_model=HealthResponse:
    return HealthResponse(status="ok")

when run directly with python:
```

```
client = TestClient(app)
call GET /health, print the status code and JSON
```

```
# practice/health_route_practice.py
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel

app = FastAPI()

class HealthResponse(BaseModel):
    status: str

@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    return HealthResponse(status="ok")

if __name__ == "__main__":
    client = TestClient(app)
    # your turn: call client.get("/health") and print
    # response.status_code and response.json()
    ...
```

Then check `http://localhost:8000/docs` with uvicorn running — you should see `/health` listed with a documented `HealthResponse` schema, which the bare-dict version from Basic never showed. Last step: move the route onto an `APIRouter` in its own file (`practice/health_routes.py`) and `include_router` it — the Solution shows the finished layout.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)